

Chapter 6

Vectorization uses special instructions to efficiently use a single core to do multiple operations at once, typically on a vector as part of a loop.

- Automatic, normally the best approach
 - The compiler can output to see which loops are vectorized, but hard to tell how effective without explicitly forcing it
- Hints
 - OpenMP can be used to hint that vectorization should be used using `#pragma` statements
 - EX: `#pragma omp simd reduction(+:variable)`
- Forced
 - Functions to vectorize can be used directly
 - `#include <x86intrin.h>`
 - This option is not portable or adaptable. Use only when absolutely required.
- Assembly
 - The raw cpu instructions may also be used if the CPU supports them and we are willing to write assembly code

Best practices for vectorization

- Restrict attribute on pointers lets vectorization occur automatically easier
- Pragma can be used when automatic vectorization is not used and it should be
- Unrolling loops may prevent a better version of vectorization
- Error checking must be done in a separate loop to allow vectorization to occur
- Inner loop should operate on the long data structure (SOA better than AOS)
- Simple loops
- Local variables inside loop
- Don't read and write to a variable inside loop
- Better to waste memory than reuse a variable (compiler might not realize safe to vectorize)